MiniShop

[![CI GitHub Actions](https://github.com/jojo8356/SAE-302/actions/workflows/ci.yml/badge.svg)](https://github.com/jojo8356/SAE-302/actions/workflows/ci.yml)

MiniShop est une application Web de boutique en ligne réalisée dans un cadre pédagogique : catalogue, panier, commandes et back-office. Le projet est développé en **PHP** et propose un stockage JSON par défaut ainsi qu'un pilote SQL via PDO.

Cadre - SAE - BUT Informatique (BUT2), Université Côte d'Azur.

Périmètre : maquette scolaire. Les paiements et les livraisons sont simulés. Le formulaire de contact n'envoie pas de courriel. Le projet n'est pas destiné à traiter de vraies ventes ni à être déployé tel quel en production.

Équipe, responsabilités et accès

Le groupe de réalisation comprend deux étudiants. Les permissions du dépôt GitLab IUT sont présentées séparément des responsabilités de planification.

Membre	Accès au dépôt GitLab IUT
Johan Polsinelli	`Owner`
Chahd Gouiaa	`Developer`

La WBS et la todolist utilisent des lettres A/B/C pour les responsabilités de projet et le RACI. Aucune correspondance entre ces lettres et les personnes nommées n'est établie dans ce README. Les permissions GitLab (`Owner`, `Developer`) ne sont pas des rôles RACI. La charte [`docs/20-charte-gitlab-revues-croisees.md`](docs/20-charte-gitlab-revues-croisees.md) décrit le processus de revue attendu ; ce README n'atteste pas que des MR ou des revues croisées ont déjà eu lieu.

Encadrement - NGUYEN Thanh-Phuong, enseignante référente, est membre directe du projet GitLab IUT avec le rôle `Reporter` ; elle ne fait pas partie du groupe étudiant.

Fonctionnalités

- **Catalogue public** : catégories, recherche, filtres, fiches produit et gestion du panier.
- **Espace client** : inscription, connexion, coordonnées, commandes et annulation selon leur état.
- **Back-office** : gestion des produits, catégories, stocks, commandes et paramètres de la boutique.
- **Préférences personnelles** : interface en français ou en anglais et fuseau horaire choisi manuellement ou estimé localement à la demande.
- **Interface responsive** et pages d'information adaptées au périmètre pédagogique.

Les décisions d'interface s'appuient sur l'étude de design de docs/DA.md. Les contrôles d'accès par rôle, protections CSRF, requêtes préparées et politique CSP sont décrits dans le code et les documents de sécurité.

Technologies et données

Domaine	Technologies / choix
Application	PHP **8.2 ou plus récent**, architecture MVC, HTML, CSS et JavaScript natif
Stockage par défaut	Fichiers JSON, sans serveur de base de données
Stockage SQL optionnel	MySQL **8.0.16+** ou MariaDB **10.6+**, via PDO (`pdo_mysql`)
Base de données	Schéma relationnel, procédures stockées, déclencheurs et migrations dans `sql/`
Tests	Suites PHP et Python ; contrôles complémentaires dans GitHub Actions

Le pilote JSON est celui utilisé par défaut. Le mode SQL est sélectionné dans `.env` avec `STORAGE_DRIVER=sql`. Les secrets et réglages propres à une machine doivent rester dans `.env`, qui n'est pas destiné à être versionné ; seul le modèle `.env.example` est suivi par Git. **Node.js et npm ne sont pas nécessaires pour lancer l'application.**

Lancer l'application en local

Prérequis

- PHP **8.2+** avec les extensions `json` et `mbstring` ;
- Bash pour `run.sh`.

L'extension `pdo_mysql` et un serveur MySQL/MariaDB sont nécessaires uniquement pour utiliser le stockage SQL. Python 3 est requis pour lancer l'ensemble des suites PHP avec le runner présenté ci-dessous.

Démarrage rapide

```
git clone https://github.com/jojo8356/SAE-302.git
cd SAE-302
./run.sh
```

Au premier lancement, `run.sh` prépare le fichier local `.env` s'il est absent, initialise le jeu de démonstration JSON si nécessaire, puis démarre le serveur PHP de développement. Ouvrir ensuite <http://localhost:8000/>.

Pour choisir un autre port :

```
PORT=8080 ./run.sh
```

Pages utiles :

- Boutique : <http://localhost:8000/>
- Catalogue : <http://localhost:8000/catalogue>
- Connexion client : <http://localhost:8000/connexion>
- Connexion administration : <http://localhost:8000/admin/connexion>
- Préférences : <http://localhost:8000/preferences>

Le serveur PHP intégré est réservé au développement local. Ne pas l'exposer directement à Internet.

Comptes de démonstration

Les comptes du seed sont réservés aux environnements locaux isolés. Les adresses, mots de passe et hashes ne sont pas reproduits dans la documentation ni dans les supports. Pour une recette LineServ, utiliser uniquement un compte de test autorisé, avec les accès transmis séparément par canal privé ; ne pas réutiliser les comptes du seed sur une instance de production ou accessible au public.

Configuration et mode SQL

Pour personnaliser la configuration locale, copier le modèle puis modifier `.env` :

```
cp .env.example .env
```

Les paramètres disponibles incluent le pilote (`STORAGE_DRIVER`), le chemin des données JSON et, pour SQL, les coordonnées de connexion (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SOCKET`). Garder les mots de passe et autres secrets hors du dépôt.

Le mode JSON ne demande aucun service supplémentaire. Pour essayer le mode SQL, installer et démarrer localement un serveur MySQL/MariaDB ainsi que l'extension PHP `pdo_mysql`, configurer l'accès dans `.env`, puis utiliser :

```
./run.sh --db
./run.sh
```

`./run.sh --db` crée la base configurée et bascule le pilote local sur SQL ; le second lancement démarre l'application. À utiliser uniquement sur une base locale dédiée aux essais. Les commandes de réinitialisation et les scripts SQL de création peuvent supprimer des données : ne jamais les exécuter sur une base partagée ou à conserver. Les détails du schéma sont dans [`docs/04-conception-bd-et-sql.md`](docs/04-conception-bd-et-sql.md).

Tests

Suites locales sans serveur SQL ni dépendances Node

Avec PHP 8.2+ et Python 3 installés, lancer les suites PHP et les contrôles documentaires sans

accéder à une base SQL ni démarrer l'E2E :

```
php scripts/run-all-tests.php --sans-sql --sans-e2e
```

Le runner détaille les suites exécutées et signale explicitement les parties non jouées. Cette commande ne valide ni l'intégration SQL ni les parcours E2E ; ne pas les considérer comme réussis lorsqu'ils sont ignorés.

Campagnes intégrées

Le runner complet est :

```
php scripts/run-all-tests.php
```

L'intégration SQL nécessite un serveur de test isolé : elle crée et supprime une base dédiée. L'E2E PHP-WASM et certains tests JavaScript ne sont exécutés que si leurs outils optionnels sont présents. Vérifier le résumé final du runner pour distinguer les tests **joués**, **non joués** et **en échec**. Ne jamais lancer de tests SQL contre une base de données contenant des informations à conserver.

Le workflow automatisé du dépôt est défini dans [`ci.yml`](.github/workflows/ci.yml). Il couvre notamment la syntaxe PHP, les suites de tests, des contrôles de sécurité, SQL sur bases jetables et des vérifications responsive en navigateur. Cette CI GitHub ne constitue pas une preuve de pipeline GitLab IUT.

Recette manuelle

- [`docs/13-fiches-recette-uc.md`](docs/13-fiches-recette-uc.md) décrit les scénarios et leur déroulement.
- [`docs/13-suivi-recette-uc.csv`](docs/13-suivi-recette-uc.csv) sert à suivre leur exécution.
- La campagne manuelle LineServ, décrite dans [`docs/14-guide-lineserv.md`](docs/14-guide-lineserv.md), vise la base réelle prévue par le protocole ; elle n'a pas été jouée et aucune restauration n'a été effectuée. Les bases jetables de CI restent distinctes.

Un scénario ne doit être marqué comme joué qu'après son exécution réelle. Les campagnes responsives multi-largeurs sont couvertes par les tests automatisés, pas par la checklist manuelle. Le dépôt ne fournit pas de procédure automatisée de déploiement.

Documentation du projet

Document	Contenu
[Cahier des charges](docs/01-cahier-des-charges-MiniShop.md)	Besoin, exigences fonctionnelles et règles métier
[Tests et validation](docs/02-document-tests-validation.md)	Stratégie, résultats et protocole de recette

[Architecture logicielle](ARCHITECTURE.md)	Couches, parcours HTTP, règles métier, persistance, sécurité, tests et limites
[Stack technique](STACK.md)	Langages, prérequis, bibliothèques et outils par contexte d'utilisation
[Conception de la base](docs/04-conception-bd-et-sql.md)	Modèle de données et choix SQL
[Stratégie de tests](docs/11-strategie-tests.md)	Périmètre et limites des tests automatisés
[Étude de design](docs/DA.md)	Analyse concurrentielle et direction visuelle
[Sécurité](docs/07-note-cybersecurite.md)	Mesures mises en place et limites connues
[Informations juridiques](docs/12-information-juridique.md)	Cadre légal de la maquette pédagogique
[Conservation des données](docs/18-politique-conservation.md)	Politique proposée et limites des outils de la maquette

Organisation du dépôt

```
app/          contrôleurs, services, dépôts, sécurité et vues
public/       point d'entrée HTTP, feuilles CSS, JavaScript et visuels
data/         données du moteur JSON
docs/         cahier des charges, conception, tests et livrables documentaires
sql/          schéma, procédures, déclencheurs, données de démonstration et migrations
scripts/      lancement, configuration, tests et outils de développement
tests/        tests PHP, Python, JavaScript et scénarios SQL
```

Licence et réutilisation

MiniShop est un projet pédagogique de SAE en BUT Informatique à l'IUT Nice Côte d'Azur. Le dépôt ne fournit pas de licence libre distincte ; toute réutilisation en dehors du cadre pédagogique doit être autorisée par les ayants droit.